Protocol-Level Optimization and Evaluation of EchoSync: A Broadcast-Based Data Synchronization Framework for Resource-Constrained LoRa Networks

[Student name] · [School, City, Country] · ISEF category: Systems Software (SOFT)

ABSTRACT Nodes in off-grid LoRa networks must keep shared datasets consistent, but pairwise synchronization scales linearly with node count under strict 1% duty-cycle limits. EchoSync lets one elected node broadcast a single stream of Rateless Invertible Bloom Lookup Table (RIBLT) coded symbols from which every receiver decodes its own, different set difference. We implemented the RIBLT codec, a LoRa airtime and duty-cycle model, Gilbert–Elliott packet loss and a radio energy model in a discrete-event simulator, validated against the Semtech airtime formula and published RIBLT overhead, and ran 2,782 simulations. Two design flaws surfaced and were corrected. First, a single sender's stream cannot deliver records that only receivers hold, so the original one-phase design left 0% of nodes fully synchronized; a two-phase protocol (broadcast, overheard upload, catch-up poll) reached 100%. Second, RIBLT has a heavy tail under frame loss: at 30% loss, receivers that missed the early dense symbols drove the stream to 20.7 coded symbols per differing record instead of 1.6. A 64-frame NACK-driven prefix repair cut airtime 3.2× and timeout recoveries 115×. At 10 nodes, EchoSync used 88% less airtime and converged 93% faster than sequential pairwise RIBLT; at 100 nodes, 99% less airtime (3.2 h vs 45 days). Convergence was 100% at every tested loss rate up to 30%, and a last-writer-wins CRDT reached identical state in 300/300 runs. Without loss, a broadcast lasted 1.01× the slowest single session (H1 supported); under 10% loss it lasted up to 1.7×. Hardware validation on ESP32/SX127x nodes is the next step.

Keywords: LoRa, LPWAN, set reconciliation, RIBLT, CRDT, broadcast protocols, duty cycle, discrete-event simulation

1. Introduction

Edge devices that operate without stable internet, such as field sensors, disaster-response kits and rural community networks, often need to agree on a shared dataset: a log of readings, a message board, a list of alerts. Conventional synchronization uses either pairwise sessions or network-wide flooding. On LoRa physical layers both are expensive. Every transmission spends scarce airtime, and European regulations cap each device at 1% of the time in the common 868 MHz sub-band [14, 21]. Pairwise synchronization needs a number of sessions that grows linearly with the number of nodes, and flooding repeats data that most receivers already hold.

EchoSync replaces many sessions with one broadcast. An elected sender streams rateless coded symbols derived from Rateless Invertible Bloom Lookup Tables (RIBLT) [1]. Because the coded symbols are linear in the sender's set, each receiver subtracts its own local symbols and decodes its *own* difference from the *same* stream, however far behind it is. The original proposal described this idea and an evaluation plan. This paper implements and evaluates it, and reports what the evaluation changed.

> **Contributions.** (1) A demonstration that one-phase broadcast reconciliation cannot reach the union of all nodes' data, and a two-phase protocol that provably can. (2) A previously unreported heavy-tail failure of RIBLT under frame loss, and a *prefix repair* mechanism that removes it. (3) An open, tested simulator that includes a full RIBLT codec, the Semtech airtime model, duty-cycle enforcement, bursty loss and an energy model. (4) A quantitative test of the original three hypotheses across 2,782 simulated runs, with the verdicts reported as measured, including where a hypothesis failed.

2. Scientific Background

2.1 LoRa PHY and LPWAN constraints

LoRa uses chirp spread spectrum: a spreading factor SF ∈ {7…12} trades data rate for range. The time on air of one frame with a payload of PL bytes, bandwidth BW, coding rate CR (1 = 4/5), an 8-symbol preamble and explicit header with CRC is [9]:

$$T_{sym} = 2^{SF}/BW, \quad T = (n_{pre} + 4.25)\cdot T_{sym} + [8 + max(\lceil (8PL - 4SF + 44)/(4(SF - 2DE)) \rceil \cdot (CR + 4), 0)]\cdot T_{sym} \quad (1)$$

Here DE = 1 (low-data-rate optimization) when T_{sym} > 16 ms. A device that transmits for T seconds must stay silent for T(1/δ − 1) seconds, where δ is the duty-cycle limit (δ = 1% in the EU868 g1 sub-band). Table 1 shows the frame sizes and airtimes used in this study. The table makes the core constraint concrete: at SF9, one full frame costs 0.62 s of airtime and then 61 s of silence.

Table 1. Frames used by EchoSync (125 kHz, CR 4/5). Payload limits follow the LoRaWAN EU868 regional parameters [15].

SF	Max payload (B)	Symbols / frame	Symbol frame (B)	Airtime (ms)	Silence at 1% (s)	Records / data frame
7	222	8	216	343.3	34.0	13
9 (default)	115	4	112	615.4	60.9	6
12	51	1	34	1810.4	179.2	2

2.2 Set reconciliation and RIBLT

Set reconciliation lets two parties learn the symmetric difference of their sets, of size d, using communication proportional to d rather than to the set size [4]. Invertible Bloom Lookup Tables (IBLTs) [2, 3] achieve this with a table whose size must be chosen in advance from an estimate of d. RIBLT [1] removes the estimate. Each item maps to coded-symbol index 0 and then to a pseudo-random sequence of indices that grows sparser (the probability of mapping to index i is about 1/(1 + i/2)). The sender can therefore emit an endless stream, and the receiver stops as soon as peeling decoding succeeds. A coded symbol stores the XOR of the items, the XOR of their 64-bit hashes, and a signed count. Yang et al. report that the number of coded symbols needed approaches 1.35·d for large d, with a higher ratio for small d [1].

The property EchoSync relies on is **linearity**. A receiver that subtracts the symbols of its own set from the sender's symbols obtains exactly the coded symbols of the difference. One stream therefore serves receivers with completely different gaps, and the stream only needs to be as long as the largest gap requires.

2.3 Conflict-free replicated data types

EchoSync synchronizes a grow-only set of immutable *operations*. Merging two such sets by union is commutative, associative and idempotent, so any state-based CRDT [5] defined as a deterministic fold over the operation set converges once the sets are equal. In our evaluation the fold is a last-writer-wins map ordered by Lamport timestamp [17] and then node id.

3. Related Work and Research Problem

Gossip and tree dissemination in wireless sensor networks, notably Trickle [6], suppress redundant transmissions by listening before speaking. They detect inconsistency cheaply, but they still ship data per item and per neighbor. Digital-fountain approaches [7, 8] showed that rateless codes make one broadcast useful to receivers with different losses. EchoSync applies the same idea to receivers

with different *data*. Vector clocks and Merkle-tree anti-entropy are standard in distributed databases, but they need several round trips, which costs minutes per exchange under a 1% duty cycle. Studies of LoRa capacity [11–13] show that collisions and duty cycle, not bit rate, are the limiting factors at scale. That motivates minimizing the number of transmissions rather than their size. Rateless reconciliation has been adopted for local-first collaboration software [18, 20], but it has not been evaluated on ultra-constrained broadcast media.

Deploying broadcast set reconciliation on LoRa raises four problems: (1) receivers with very different state gaps; (2) frame loss on fading channels, which causes local decoding failures; (3) convergence verification without acknowledgment storms; and (4) collisions among control and request frames. Section 5 addresses each one.

4. Research Questions and Hypotheses

RQ1. Does broadcasting rateless coded symbols reduce total network airtime compared with sequential pairwise synchronization as node density varies?
RQ2. How do heterogeneous state gaps among receivers affect broadcast duration and airtime efficiency?
RQ3. Can EchoSync converge robustly under 10–30% packet loss without excessive control overhead?

The hypotheses are kept exactly as originally stated. Because some of them were ambiguous, we fixed operational definitions *before* running the final experiments:

- **H1:** broadcast execution time $\leq$ 1.1× the largest pairwise session, and total synchronization time reduced by $\geq$ 50% compared with sequential pairwise sync. *Measured as* the wall time of phase 1 divided by the wall time of the slowest single-receiver session run with the identical mechanism from the same initial state (H1a), and $1 - T_{\text{EchoSync}}/T_{\text{pairwise}}$ to full convergence (H1b).
- **H2:** the number of transmitted coded symbols approaches the theoretical minimum within a 15% margin. The phrase "theoretical minimum" is ambiguous, so we report both readings: (i) the information bound $d_{\max}$, the largest receiver gap; and (ii) the symbols a unicast RIBLT session needs for the same d (the minimum for *this code*).
- **H3:** convergence succeeds under 10–30% loss when supplementary symbol requests are used. *Measured as* the fraction of runs in which every node holds the union and all fingerprints match before a 120-day simulated timeout.

5. Protocol Design

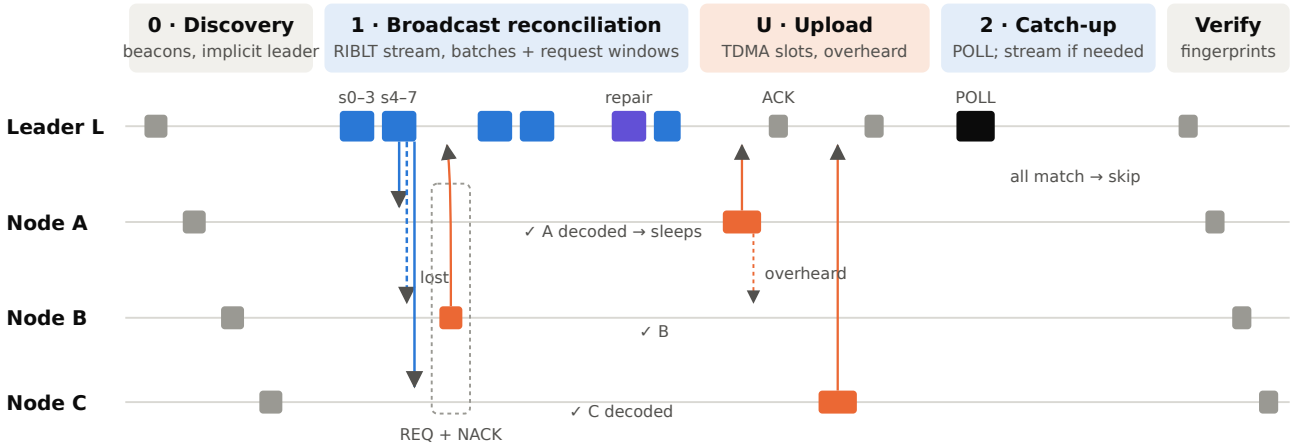


Figure 1. EchoSync phases. Phase 1: the leader streams RIBLT symbols in batches; after each batch, receivers that have not decoded send a request (REQ) in a short contention window, and the NACK bitmap in the request triggers retransmission (repair) of lost early frames. Receivers that decode sleep. Phase U: receivers upload, in roster order, the records the leader lacks; other nodes overhear them. Phase 2: a POLL carries the union fingerprint, and only nodes that are still behind request a catch-up stream.

5.1 Data model and frames

Each record (CRDT operation) is a fixed 16-byte item, so a coded symbol carries the record itself in its sum field and no separate data-transfer round is needed. A coded symbol is 26 bytes on air: a 16 B sum, an 8 B checksum, and a 2 B count stored mod 2^{16}, which is enough to recognize ±1. Table 2 lists the frame formats.

Table 2. Frame formats (bytes). Every frame starts with an 8 B header: type, sender id (2), session (1), first symbol index (3), count (1).

Frame	Size	Contents
BEACON	20	header, set size (4), 64-bit state fingerprint (XOR of record hashes)
SYMBOLS	8 + 26K	K consecutive coded symbols (K = 8 / 4 / 1 at SF7 / 9 / 12)
REQ (+NACK)	8 (+8)	"need more symbols", plus a 64-bit bitmap of lost frames among the first 64
DATA / ACK	8 + 16r / 9	r packed records / 8-bit bitmap of received frames in a burst
POLL	20	union size and fingerprint after the upload phase

5.2 Discovery and leader election without messages

Every node sends one beacon. The leader is chosen deterministically from what every node has already heard: the largest set, with ties broken by the lowest id. No election messages, rounds or votes are needed. If all fingerprints match, nothing is sent. Beacons are periodic in deployment; the simulator assumes the roster is complete, which is a stated limitation (Section 9).

5.3 Phase 1: broadcast reconciliation

The leader streams symbol frames in batches of B frames (default B = 2). The first batch is sized from the beacons' set-size gaps ($\lceil 1.35 \cdot \max|\Delta\text{Size}|/K \rceil$ frames), which helps a node that joins late with an empty set. Each receiver feeds received symbols into a streaming peeling decoder. It recognizes completion by **fingerprint verification**: the XOR of the hashes of recovered items must equal fp(S) ⊕ fp(R), and the signed count must equal |S| − |R|. Both values come from beacons, so a receiver needs no "end" message and loses nothing if symbol 0 is dropped. A receiver that has decoded stops listening (**early sleep**).

5.4 Request windows, collisions and prefix repair

After each batch the leader listens for a short window of four slots. Every receiver that has not decoded sends a REQ in a random slot. The leader only needs to detect *activity*, using LoRa Channel Activity Detection, so collisions never stop the stream; this solves the acknowledgment-storm problem by design. If no request is heard but a receiver is still behind, that receiver retries after 30 s, and the stream resumes from where it stopped. Because the code is rateless, no symbol is wasted.

Prefix repair. Early experiments showed a failure mode that the original proposal did not anticipate. RIBLT places every item in symbol 0 and concentrates mappings in the first indices. A receiver that loses those dense early frames must wait for the increasingly sparse later symbols, which happen to contain its remaining items only rarely. With 80 missing items and 20% frame loss, the median receiver needed 1.85·d symbols, but the 99th percentile needed 6.9·d and the worst case 427·d. We therefore extended REQ with a 64-bit NACK bitmap of lost frames among the first 64. A request that lands alone in its slot is decoded, and the leader retransmits the union of the reported frames before the next batch. Retransmissions help every lagging receiver at once. With repair, the same test had a 99th percentile of 1.9·d.

5.5 Why one phase cannot converge, and phases U and 2

> **Observation.** Let S be the leader's set and R_i the receivers' sets. After phase 1, receiver i holds $S \cup R_i$. A record $x \in R_i \setminus S$ can never reach receiver $j \neq i$, because the stream encodes only S. All nodes reach the union $U = \bigcup R_i$ only if $R_i \subseteq S$ for every i, that is, only if the leader already holds everything. In any network where nodes generate their own data this condition fails, and the original single-phase design never converges.

Decoding phase 1 tells each receiver exactly which of its records the leader lacks (the items with negative sign). In **phase U**, receivers upload those records in roster order, using reliable packed DATA frames with ACKs. Other receivers overhear the uploads and skip records they have already heard, so each record usually crosses the channel once. In **phase 2**, the leader sends a POLL with the union fingerprint. Nodes that missed some uploads request, and only they receive a catch-up stream; if nobody requests, phase 2 costs one frame. A final beacon round verifies that all fingerprints are equal.

6. Methodology

6.1 Simulator

We wrote a discrete-event simulator in Python (about 1,100 lines, five files; Appendix A). It contains:

- A byte-exact RIBLT encoder and streaming decoder that follows the reference design [1, 19], with BLAKE2b-64 hashing and the reference index-mapping generator. It accepts out-of-order (repaired) symbols.
- Equation (1) for airtime.
- Per-device duty-cycle enforcement: after each frame, the device is silent for $T(1/\delta - 1)$.
- A shared single-hop channel with one transmission at a time.
- Per-link frame loss, either Bernoulli or Gilbert–Elliott bursts [16] with a mean burst length of 4 frames.
- Radio energy from the SX1276/77/78 datasheet [10]: 87 mA TX at +17 dBm, 10.8 mA RX, 3.3 V.

Receivers spend RX energy only while they are expected to listen.

6.2 Protocols compared

- **EchoSync (2-phase)**: Section 5, with repair.
- **EchoSync (1-phase)**: the original design.
- **Pairwise RIBLT (star)**: the elected leader runs sequential bidirectional RIBLT sessions with every node. The node streams symbols, the leader replies MORE (with the same NACK repair, for fairness) or sends the missing records reliably. Two passes are needed so that early nodes also receive later nodes' records. This baseline uses the same code, so the comparison isolates *broadcast versus pairwise*.
- **Full-state broadcast**: each node broadcasts its entire set once, idealized with perfect scheduling and no loss or collisions.
- **Oracle**: every record that is missing anywhere is sent exactly once. This is an information lower bound and not a real protocol.

6.3 Workload and parameters

Nodes share a history of 1,000 records. Each node misses each historical record with probability 3% and holds 5–20 fresh records of its own, so a typical gap is $d \approx 85$–100. Defaults: SF9, BW 125 kHz, CR 4/5, $\delta = 1\%$, $K = 4$, $B = 2$, 4 request slots, a 30 s recovery timeout and a 5 s unicast timeout. Each configuration was repeated with independent seeds: 20–30 seeds per point (8 at $N \geq 50$, 100 for codec points with $d \leq 100$). We report means with 95% confidence intervals (shaded bands and error bars in the figures).

6.4 Validation

Automated tests (`test_sim.py`, all passing) check the following:

- Airtime matches the Semtech calculator (SF7/20 B: 56.58 ms; SF12/51 B: 2,465.79 ms).
- Decoding full sets recovers the exact difference in both directions.
- The difference stream is bit-identical to subtracting two full-set streams, which justifies simulating receivers by their differences.
- Overhead at $d = 1{,}000$ falls in 1.30–1.45, matching [1].
- Decoding survives 30% symbol loss.
- EchoSync and the pairwise baseline converge under 0%, 20% and 30%-burst loss.

7. Results

7.1 Coding overhead (H2)

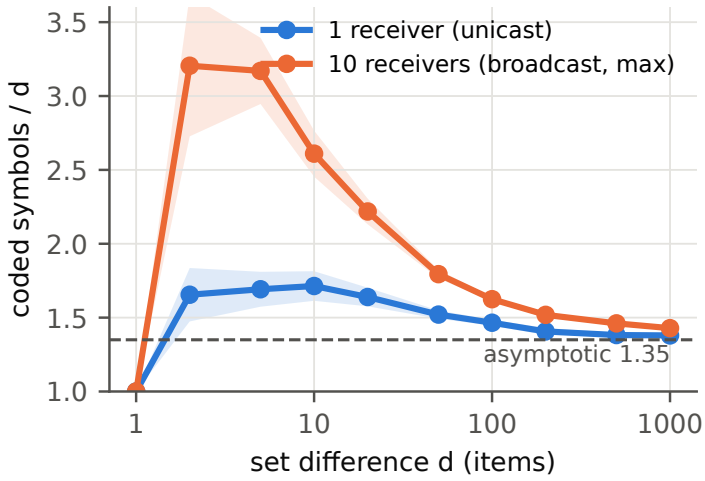


Figure 2. Coded symbols needed per differing item. The broadcast curve is the maximum over 10 receivers that each lack a different random d items from one stream. Bands show 95% confidence intervals.

Unicast overhead peaks at 1.71 for d = 10 and falls to 1.38 at d = 1,000, which reproduces [1]. A broadcast must wait for its slowest receiver, so its overhead is higher at small d (2.61 at d = 10) but within 10.8% of unicast at d = 100 and within 3.6% at d = 1,000 (Table 3). Measured against the information bound d, the overhead never comes within 15% (the minimum is 1.38). **H2 is rejected in reading (i). In reading (ii) it is supported for d ≥ 100 but not for small differences.**

Table 3. Mean coded symbols per differing item, by set difference d.

d	10	20	50	100	200	500	1000
Unicast (1 receiver)	1.71	1.64	1.52	1.47	1.41	1.38	1.38
Broadcast (10 receivers)	2.61	2.22	1.79	1.63	1.52	1.46	1.43
Broadcast vs unicast	+52%	+35%	+18%	+10.8%	+8.0%	+5.7%	+3.6%

7.2 Node scaling (RQ1, H1)

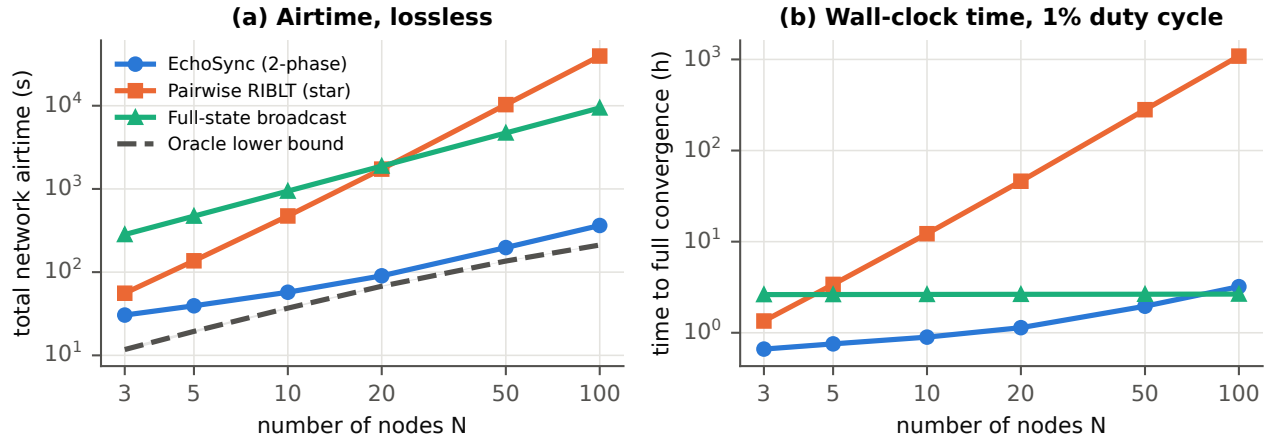


Figure 3. Lossless node scaling at SF9, both axes logarithmic. (a) Total airtime of all transmitters. (b) Time until every node holds the union, under 1% duty cycle. The legend in (a) applies to both panels.

EchoSync's airtime grows slowly with N: a single stream is followed by uploads that are proportional to the number of new records. Pairwise airtime grows roughly quadratically: there are about 2N sessions, and the second-pass gaps themselves grow with N. EchoSync stays within 1.3–1.7× of the oracle lower bound for N ≥ 10 (2.6× at N = 3, where fixed costs such as beacons dominate). Full-state broadcast costs 9–26× more airtime than EchoSync. Its wall time looks short only because the idealized model lets 100 radios share their duty budgets perfectly with zero collisions, while keeping the channel 98% busy, which an ALOHA LoRa channel cannot sustain [11].

Table 4. Lossless scaling, means over seeds. "Converged" is the fraction of nodes holding the union at the end.

N	EchoSync airtime (s)	Pairwise airtime (s)	Airtime saved	EchoSync time	Pairwise time	Time saved	1-phase converged
3	30.5	55.8	45.4%	0.66 h	1.35 h	50.8%	0%
5	39.4	136.9	71.2%	0.76 h	3.39 h	77.8%	0%
10	57.3	473.4	87.9%	0.90 h	12.2 h	92.7%	0%
20	90.4	1,730.8	94.8%	1.14 h	46.0 h	97.5%	0%
50	197.4	10,307	98.1%	1.96 h	11.7 d	99.3%	0%
100	363.6	39,547	99.1%	3.21 h	45.3 d	99.7%	0%

H1a (no loss): supported. Phase 1 took 1.004–1.012× the slowest single-receiver session on average, with a worst run of 1.048×. **H1a (10% loss): not supported.** The ratio rose to 1.11–1.37× on average, with a worst run of 1.71×, because the broadcast waits for the unluckiest of N−1 lossy receivers. **H1b: supported for N ≥ 5**, with time saved of 77.8–99.7% without loss and 71.8–99.5% at 10% loss. At N = 3 the saving was 50.8% without loss but only 45.5% at 10% loss, just under the hypothesis.

7.3 Heterogeneous state gaps (RQ2)

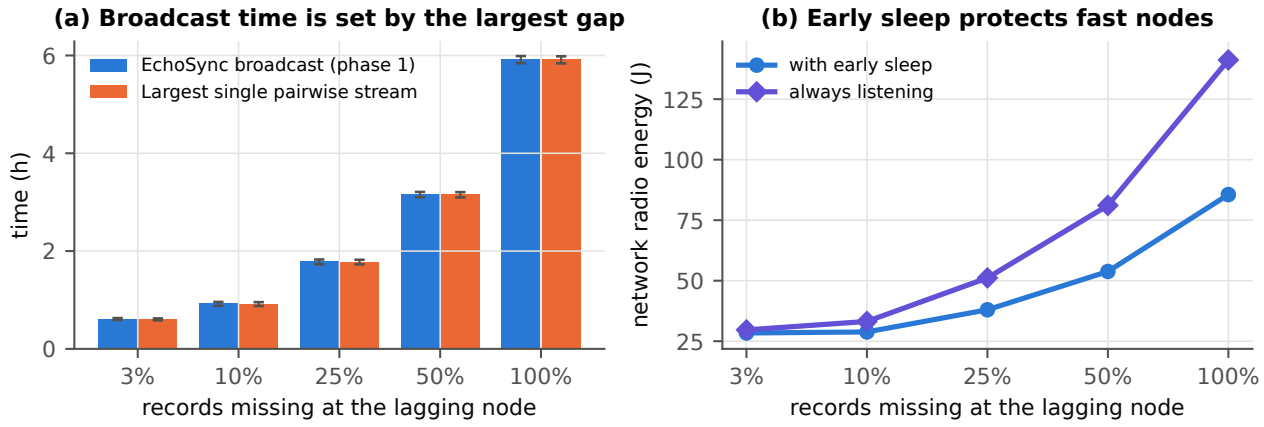


Figure 4. Ten nodes; node 9 misses a growing fraction of the shared history (100% means a new node that joins with an empty set). (a) Phase-1 broadcast time versus the slowest single pairwise stream. (b) Total radio energy with and without early sleep.

Broadcast duration is set almost entirely by the largest gap. Even for a node that joins empty (d ≈ 1,007), phase 1 lasted 1.001× the time needed to sync that node alone. The other eight receivers are served "for free" inside the same stream. Without early sleep, fast nodes would keep listening for the laggard. Early sleep cut network radio energy by 26% at a 25% gap and by 39% for an empty joiner (85.6 J vs 141.2 J). End to end, the empty joiner case converged in 6.2 h with EchoSync versus 20.6 h with pairwise sessions.

7.4 Packet loss and prefix repair (RQ3, H3)

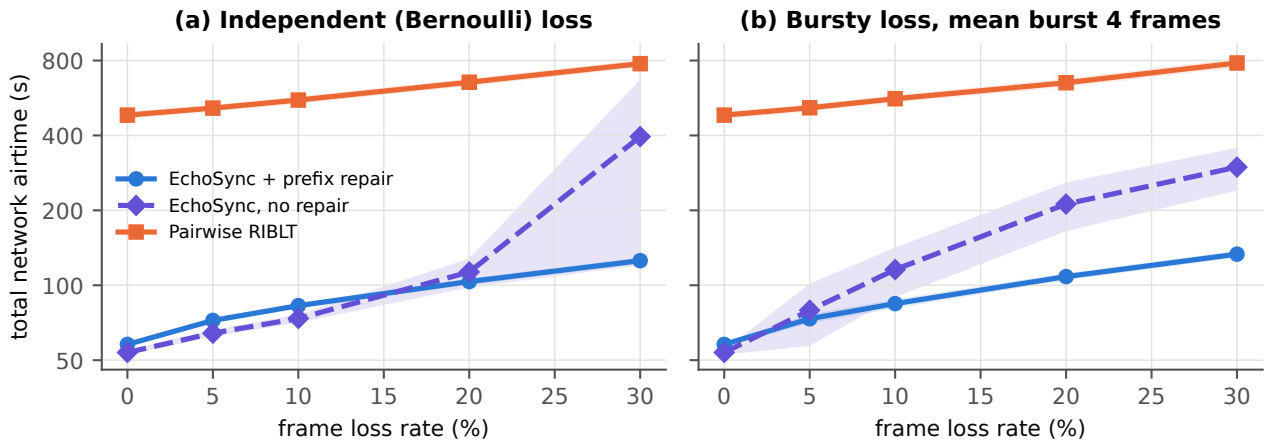


Figure 5. Total airtime with 10 nodes as frame loss grows (30 seeds per point). Prefix repair removes the heavy tail that otherwise dominates at ≥ 20% loss.

Table 5. Loss experiment (N = 10). "Symbols / d_{max}" counts new coded symbols relative to the largest gap; "recoveries" counts 30 s timeouts per run.

Loss	Airtime, repair (s)	Airtime, no repair (s)	Pairwise (s)	Symbols / d_{max} (repair / none)	Recoveries (repair / none)	Time, repair	Converged
0%	57.8	53.7	482.7	1.59 / 1.59	0 / 0	0.90 h	100%
10% iid	82.8	73.5	554.9	2.01 / 2.36	0.4 / 0.5	1.51 h	100%
20% iid	103.5	113.0	654.4	2.19 / 4.32	0.4 / 4.7	2.03 h	100%
30% iid	125.6	395.7	777.1	2.34 / 20.7	0.7 / 80.6	2.55 h	100%
10% burst	84.4	115.6	561.7	2.12 / 5.05	0.7 / 7.4	1.58 h	100%
30% burst	133.4	298.4	782.0	2.71 / 14.5	4.0 / 76.1	2.81 h	100%

H3: supported. All 300 loss runs converged, with verified fingerprints, at every rate from 0% to 30%, for both independent and bursty loss. Without repair, convergence still happened but was erratic: at 30% independent loss, the 95th-percentile airtime was 1,009 s versus 139 s with repair. Repair costs a little at low loss, because the 8-byte NACK and its slots add about 9 s at 10% loss. It pays off from 20% loss upward, and at 30% it reduced airtime 3.2× and timeouts 115×. Even at 30% loss, EchoSync used 6.2× less airtime than pairwise sync and finished in 2.6 h versus 20.3 h.

7.5 Additional experiments

Table 6. Asymmetric links, spreading factor and batch size (N = 10; means over 15–20 seeds).

Configuration	EchoSync airtime (s)	EchoSync time	Pairwise airtime (s)	Pairwise time
All links 5% loss	71.6	1.24 h	505.6	13.0 h
1 far node at 30%, rest 5%	89.4	1.69 h	567.0	14.7 h
3 far nodes at 30%, rest 5%	107.8	2.14 h	651.9	17.0 h
SF7 (short range)	14.2	0.23 h	132.2	3.33 h
SF9 (default)	57.3	0.89 h	469.2	12.1 h
SF12 (maximum range)	956.5	10.4 h	5,419	138.7 h
Batch B = 1 / 2 / 4 / 8, no loss	72.8 / 57.3 / 50.0 / 46.8	0.89–0.94 h	–	–
Batch B = 1 / 2 / 4 / 8, 20% loss	120.9 / 103.2 / 93.7 / 92.9	1.99–2.09 h	–	–

All asymmetric and spreading-factor runs converged. At SF12, the slowest and longest-range setting, a 10-node network synchronizes in about 10 hours with EchoSync. Pairwise sync takes almost 6 days, which makes it impractical. Larger batches mean fewer request windows. B = 4 was slightly better than the pre-chosen B = 2, and B = 8 sent 34% more symbols under loss. We kept B = 2 as defined before the experiments and note B = 4 as a tuning option.

CRDT convergence. In 300 runs (0%, 20% and 30% loss), 10 nodes each issued 5–30 concurrent last-writer-wins updates on 20 shared keys, about 177 operations per run with every key conflicting. In all 300/300 runs, every replica ended with an identical state that matched a centralized reference.

8. Discussion

Table 7. Hypothesis verdicts.

Hypothesis	Verdict	Evidence
H1a: broadcast ≤ 1.1× largest session	Supported without loss; not supported at 10% loss	1.01× mean (max 1.05×); 1.11–1.37× mean (max 1.71×) under loss
H1b: ≥ 50% time reduction	Supported for N ≥ 5; borderline at N = 3	77.8–99.7% saved; N = 3: 50.8% (no loss), 45.5% (10% loss)
H2: symbols within 15% of minimum	Rejected vs d; supported vs unicast RIBLT for d ≥ 100	Codec overhead ≥ 1.38; broadcast penalty +10.8% at d = 100, +52% at d = 10
H3: converges at 10–30% loss	Supported	300/300 runs; repair keeps the cost to 2.2× the lossless cost at 30%

Where the savings come from. Pairwise synchronization pays for every gap separately, and every session waits on two duty-cycle budgets. EchoSync pays roughly once for the largest gap, plus about one transmission per genuinely new record. That is why its cost tracks the oracle bound (1.3–1.7× for $N \geq 10$) instead of growing with N.

Leader hotspot. The leader transmits nearly half of all airtime (45% at N = 10), so its battery and duty budget are the bottleneck. Wall time scales as $1/\delta$. Two remedies follow directly, and both are future work: rotate leadership between rounds, which the deterministic election makes easy, and use the EU868 sub-band that allows a 10% duty cycle for the stream.

Microcontroller memory. The original paper listed SRAM as a limit without numbers. A receiver stores about 1.4·d received symbols (28 B each in RAM) plus the recovered records (16 B each). The local set's symbols can be computed on the fly from flash. A gap of d = 1,000 therefore needs about

56 kB, and a 300 kB budget on an ESP32 (520 kB SRAM) supports a gap of about 5,000 records per session. Larger gaps can be split into several sessions.

Changes made to the original design as a result of this study:

- A two-phase protocol, because one phase never converges.
- Prefix repair with NACK bitmaps.
- Collision-tolerant busy-tone request windows, which solve the acknowledgment storm.
- Fingerprint-based termination.
- Message-free leader election.
- Hardware corrected so that the radio matches the band (Section 10).
- Multi-hop moved to future work, because the single-hop design is what was evaluated.

9. Limitations

- **Simulation only.** No hardware measurements are reported yet. Radio behavior such as the capture effect, clock drift, CAD reliability and receive-to-transmit turnaround is modeled ideally or not at all.
- **Single hop.** All nodes are assumed to hear each other, apart from loss. Multi-hop relaying (1–5 hops in the original plan) was not evaluated.
- **Channel model.** Transmissions within the protocol are serialized. Interference from other LoRa networks is represented only by the loss rate. The full-state baseline is idealized in its favor.
- **Static membership during a round.** Nodes joining or leaving mid-session and records created during a session are not simulated; new records simply wait for the next round.
- **Workload.** The workload is synthetic, with uniform random gaps and fixed 16-byte records. Real data may cluster, and larger records would need fragmentation.
- **Security.** No authentication is included. A malicious node could inject records or false fingerprints. Signed records are future work.

10. Conclusion and Future Work

A single rateless broadcast can serve many receivers with different gaps at almost the cost of serving the worst one. Turning that idea into a working LoRa protocol required two corrections that only appeared through implementation and measurement: a second phase to move records *toward* the leader, and prefix repair to survive loss. With both corrections, EchoSync converged in every simulated run. It used 88–99% less airtime than pairwise RIBLT for 10–100 nodes and synchronized a 100-node network in about 3 hours instead of about 45 days.

Next step: hardware validation. Six ESP32 nodes will be built with radios matched to the band. The SX1278 covers 137–525 MHz and is therefore used at 433 MHz. The SX1276 is used at 868 MHz. The tests will measure real airtime, loss and battery current against these predictions. Further work: multi-hop clustering, leader rotation, the 10% duty-cycle sub-band, authenticated records, and nodes joining and leaving during operation.

Ethical Considerations

All planned radio experiments use licence-exempt ISM/SRD bands (433 MHz or 868 MHz) within the power and duty-cycle limits of the applicable national regulator. The study involves no human or animal subjects, and the simulated datasets contain no personal data.

Acknowledgments

[Mentor / teacher acknowledgment.] [Disclosure of any assistance received, including AI tools, as required by ISEF rules.]

References

1. L. Yang, Y. Gilad, M. Alizadeh. Practical Rateless Set Reconciliation. *Proc. ACM SIGCOMM*, 2024. arXiv:2402.02668.
2. D. Eppstein, M. T. Goodrich, F. Uyeda, G. Varghese. What's the Difference? Efficient Set Reconciliation without Prior Context. *Proc. ACM SIGCOMM*, 2011.
3. M. T. Goodrich, M. Mitzenmacher. Invertible Bloom Lookup Tables. *Proc. 49th Allerton Conference*, 2011.
4. Y. Minsky, A. Trachtenberg, R. Zippel. Set Reconciliation with Nearly Optimal Communication Complexity. *IEEE Trans. Information Theory* 49(9), 2003.
5. M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-Free Replicated Data Types. *Proc. SSS*, LNCS 6976, 2011.
6. P. Levis, N. Patel, D. Culler, S. Shenker. Trickle: A Self-Regulating Algorithm for Code Propagation and Maintenance in Wireless Sensor Networks. *Proc. USENIX NSDI*, 2004.
7. J. W. Byers, M. Luby, M. Mitzenmacher, A. Rege. A Digital Fountain Approach to Reliable Distribution of Bulk Data. *Proc. ACM SIGCOMM*, 1998.
8. M. Luby. LT Codes. *Proc. IEEE FOCS*, 2002.
9. Semtech. AN1200.13: LoRa Modem Designer's Guide, 2013.
10. Semtech. SX1276/77/78/79 Datasheet, Rev. 7, 2020.
11. M. Bor, U. Roedig, T. Voigt, J. Alonso. Do LoRa Low-Power Wide-Area Networks Scale? *Proc. ACM MSWiM*, 2016.
12. A. Augustin, J. Yi, T. Clausen, W. M. Townsley. A Study of LoRa: Long Range & Low Power Networks for the Internet of Things. *Sensors* 16(9):1466, 2016.
13. F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melià-Seguí, T. Watteyne. Understanding the Limits of LoRaWAN. *IEEE Communications Magazine* 55(9), 2017.
14. ETSI EN 300 220-2: Short Range Devices (SRD) operating in the frequency range 25 MHz to 1 000 MHz, Part 2. V3.2.1, 2018.
15. LoRa Alliance. RP002-1.0.4 LoRaWAN Regional Parameters, 2022.
16. E. N. Gilbert. Capacity of a Burst-Noise Channel. *Bell System Technical Journal* 39(5), 1960.
17. L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM* 21(7), 1978.
18. Ink & Switch. Syncing Keyhive: Rateless Invertible Bloom Lookup Tables. Lab notebook 05, 2025. inkandswitch.com/keyhive/notebook/05/
19. L. Yang. riblt: reference implementation of Rateless IBLT (Go). github.com/yangl1996/riblt
20. M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan. Local-First Software: You Own Your Data, in Spite of the Cloud. *Proc. ACM Onward!*, 2019.
21. CEPT/ERC. Recommendation 70-03: Relating to the Use of Short Range Devices (SRD), latest amendment.

Appendix A. Reproducibility

The simulator has five Python files:

- `riblt.py`: codec.
- `sim.py`: PHY, channel and protocols.
- `experiments.py`: all 2,782 runs, about 9 minutes on 2 CPU cores.
- `analyze.py`: figures and summary statistics.
- `test_sim.py`: validation tests.

The only dependencies are numpy and matplotlib. Run `python test_sim.py && python experiments.py && python analyze.py`. Every random choice is seeded, so all figures and tables can be regenerated bit-for-bit.